

TMDB Discover – Functional Test Step Document

1. Document Information

Field	Details
Application	TMDB Discover Demo
Application URL	https://tmdb-discover.surge.sh/
Application Type	Movie / TV Show Listing Web Application
Test Type	Functional, UI, Negative, Boundary, Integration
Prepared By	QA Engineer
Test Environment	Web
Browsers	Chrome / Firefox / Edge
Automation Candidate	Yes – Playwright + Python + Pytest

2. Test Strategy

The objective of this testing is to verify that users can discover Movies and TV Shows by using the available filtering, sorting, search, and pagination functionality.

The major areas covered are:

- Categories: Popular, Trending, Newest, Top Rated
- Title search/filter
- Type: Movies / TV Shows
- Year of Release
- Rating
- Genre
- Pagination
- URL/slug navigation
- Invalid and boundary conditions
- Combination of multiple filters

The application is a JavaScript-based demo application.

Detailed Test Cases

001 - Verify Popular Category

Module: Category Filter

Priority: High

Type: Positive

Automation: Yes

Objective

Verify that selecting the **Popular** category displays content belonging to the Popular category.

Preconditions

1. Browser is available.
2. Internet connectivity is available.
3. Application is accessible.
4. User is on the TMDB Discover home page.

Test Data

Category = Popular

Test Steps

Step	Action	Expected Result
1	Open https://tmdb-discover.surge.sh/	Application should load successfully
2	Wait until movie/TV cards are displayed	Content cards should be visible
3	Get the api response of “ **/movie/popular ”	• The status should success, • The response data should follow the schema. • The Order of the response - > Result item should in descending order of the popularity

4	Validate Default page Popular	Popular option should become selected
5	Observe the displayed content	Content should refresh/update
6	Verify that results are displayed	At least one result should be visible
7	Verify title, image and available metadata on cards	Card information should render correctly
8	Compare the UI Movie card order with api result movie Title order	The order of the api response movie title and UI movie title should be equal

Expected Overall Result

The application should display Popular Movies/TV Shows without UI/api errors.

002 - Verify Trending Category

Test Data

Category = Trending

Step	Action	Expected Result
1	Launch the application	Home page should load
2	Select Trending	Trending should become active
3	Get the api response of <code>**/movie/trend</code>	- The status should success, - The response data should follow the schema.
4	Wait for results to refresh	Loading should complete

5	Verify content cards	Trending results should be displayed
6	Verify no duplicate or broken cards	Results should render correctly
7	Select another category and return to Trending	Trending results should load consistently

Note: unable to find the sorting logic so validated by changing categories and the sort of movie card should change

003 - Verify Newest Category

Priority: High

Type: Positive

Steps

Step	Action	Expected Result
1	Open application	Application loads
2	Select Newest	Newest category is selected
3	Get the api response of <code>**/3/movie/now_playing</code>	• The status should success, • The response data should follow the schema. • There is date with min and max date the result should be contained within the date

4	Observe the displayed content	Content should refresh/update
5	Verify that results are displayed	At least one result should be visible
6	Verify title, image and available metadata on cards	Card information should render correctly
7	Compare the UI Movie card order with api result movie Title order	The order of the api response movie title and UI movie title should be equal

004 - Verify Top rated Category

Priority: High

Type: Positive

Steps

Step	Action	Expected Result
1	Open application	Application loads
2	Select Top rated	Top rated category is selected
3	Get the api response of <code>**/3/movie/top_rated</code>	• The status should success, • The response data should follow the schema. • The Order of the response - > Result item should in descending order of the <code>vote_average(with one decimal value)</code>

4	Observe the displayed content	Content should refresh/update
5	Verify that results are displayed	At least one result should be visible
6	Verify title, image and available metadata on cards	Card information should render correctly
7	Compare the UI Movie card order with api result movie Title order	The order of the api response movie title and UI movie title should be equal

005 - Search/Filter Using Valid Title

Priority: High

Type: Positive

Test Data

Title = Avengers

Steps

Step	Action	Expected Result
1	Launch application	Application loads
2	Locate Title filter/search field	Field should be enabled

3	Enter <i>Avengers</i>	Text should appear correctly
4	Apply filter/search if required	Search should execute
5	Observe results	Matching titles should be returned
6	Inspect displayed names	Returned titles should match/relevantly contain <i>Avengers</i>
7	Clear Title filter	Original/default results should return

006 - Search/Filter Using Invalid Title

Priority: Medium

Type: Negative

Test Data

Title = testinvalidsearch

Steps

Step	Action	Expected Result
1	Open application	Application loads
2	Enter invalid/non-existing title	Value is accepted
3	Apply search/filter	Request is processed

- | | | |
|---|-----------------------------|---|
| 4 | Observe results | No unrelated content should appear |
| 5 | Verify empty-state behavior | Meaningful "No results" state should be displayed |
| 6 | Verify page remains usable | No crash or broken layout should occur |

007 - Filter Movies Types

Test Data

Type = Movies

Step	Action	Expected Result
1	Open application	Application loads
2	Select Type = TV Shows	Default select is Movies go to tv then select Movies
3	Select Type = Movies	Movies option becomes active
4	Get api response	The request url should be contain movie
5	Wait for result refresh	Results load

6	Inspect several results	Results should represent Movies
7	Navigate to next page	Movie filter should remain applied
8	Navigate back	Movie filter should still be active

008 - Filter TV ShowsTypes

Test Data

Type = Movies

Step	Action	Expected Result
1	Open application	Application loads
2	Select Type = TV Shows	TV Shows becomes active
3	Get api response	The request url should be contain tv
4	Wait for result refresh	Results load

5	Inspect several results	Results should represent Movies
6	Navigate to next page	Movie filter should remain applied
7	Navigate back	Movie filter should still be active

009 - Filter by Valid Year range

Test Data

Year = 2000 - 2010

Step	Action	Expected Result
1	Open application	Application loads
2	Locate Year filter	Year control is available
3	Select/enter 2000 - 2010	Value is accepted
4	Apply filter	Results refresh

- | | | |
|---|--|--|
| 5 | Inspect release year of returned results | <ul style="list-style-type: none">● UI: Results should correspond to movie release year should be between 2000 -2010● API: Results should correspond to movie release year should be between 2000 -2010 |
|---|--|--|

0010 - Filter by Rating

Test Data

Rating range = 4 to 5

Steps

Step	Action	Expected Result
1	Open application	Application loads
2	Set Rating 4 to 5	Rating value is applied
3	Wait for result refresh	Results load
4	Inspect displayed ratings	Results should satisfy configured rating behavior
5	Check multiple results	Filter should be consistently applied

6 Remove rating filter Results should reset

0011 - Filter by Genre

Test Data

Genre = Comedy

Step	Action	Expected Result
1	Open application	Application loads
2	Open Genre filter	Available genres are displayed
3	Select Comedy	Action becomes selected
4	Apply filter if required	Results refresh
5	Inspect returned titles	Results should belong to/relevantly match Action genre
6	Navigate to next page	Genre selection should remain
7	Clear filter	Default results should return

0012 - Combination of Multiple Filters

This is particularly important because individual filters may work while combinations fail.

Test Data

Category = Top Rated

Type = Movies

Year = 2000 -2010

Rating = 4 to 5

Genre = Comedy

Steps

Step	Action	Expected Result
1	Open application	Application loads
2	Select Top Rated	Filter applied
3	Select Movies	Filter applied
4	Select year 2000-2010	Filter applied
5	Set rating 4 to 7	Filter applied
6	Select Comedy genre	Filter applied
7	Observe results	Results should satisfy the combination of supported filters

8 Navigate to page 2 All selected filters should remain applied

0013 - Verify First Section pagination Functionality

Priority: High

Step	Action	Expected Result
1	Open application	Page 1 is displayed In api the payload contain <code>page=1</code>
2	Get the loaded movie list title, add to the know movie list	Titles are captured
4	Click Page 2	• Page 2 to be loaded • In api the payload contain <code>page=2</code>
5	Get the loaded movie list title from page 2, add to the know movie list	Validated the movie title loaded not match with page 1 movie title
6	Click Page 3	• Page 3 to be loaded • In api the payload contain <code>page=3</code>

- | | | |
|---|---|---|
| 7 | Get the loaded movie list title from page 3 | Validated the movie title loaded not match with page 1,2 movie title with know movie list |
| 8 | Check page 4 is visible,Click Page 4 | <ul style="list-style-type: none"> ● Page 4 to be loaded ● The Page no 4 is visible |
| 9 | Get the loaded movie list title from page 4 | Validated the movie title loaded not match with page 1,2,3 movie title with know movie list |

0014 - Verify Last Section pagination Functionality

Priority: High

- | Step | Action | Expected Result |
|------|-------------------------------|---|
| 1 | Open application | Page 1 is displayed

In api the payload contain
page=1 |
| 2 | Get the api response | Validate total_pages match the last page number in ui |
| 4 | Click Page last number | <ul style="list-style-type: none"> ● Page Last to be loaded ● In api the payload contain page={last page num} |

- | | | |
|---|---|--|
| 5 | Get the loaded movie list title from {last page number}, add to the know movie list | Titles are captured |
| 6 | Click last page number - 1 | <ul style="list-style-type: none">• last page number - 1 to be loaded• In api the payload contain { last page number - 1} |
| 7 | Get the loaded movie list title from last page number - 1 | Validated the movie title loaded not match with last page movie title with know movie list |
| 8 | Click last page number - 2 | <ul style="list-style-type: none">• last page number - 2 to be loaded• In api the payload contain { last page number - 2} |
| 9 | Get the loaded movie list title from last page number - 2 | Validated the movie title loaded not match with last two page movie title with know movie list |
| 8 | Check last page number - 3 is visible,Click last page number - 3 | <ul style="list-style-type: none">• last page number - 3 to be loaded• Validate The last page number - 3 is visible |

- | | | |
|---|--|--|
| 9 | Get the loaded movie list title from last page number - 3 | Validated the movie title loaded not match with last three page movie title with know movie list |
|---|--|--|

0015 - Verify Middle Section pagination Functionality

Priority: High

Step	Action	Expected Result
1	Open application	Page 1 is displayed In api the payload contain <code>page=1</code>
2	Get the loaded movie list title, add to the know movie list	Titles are captured
4	Click Page (...)	• Page ... to be loaded • In api the payload contain <code>page=default_page+3</code> • Validate UI the page number is default_page+3
5	Click Page (...)	• Page ... to be loaded • In api the payload contain <code>page=default_page+3+3</code>

- Validate UI the page number is $\text{default_page}+3+3$
 - Validate two “...” is displayed
- 6 Click on the second **Page (...)**
- Page ... to be loaded
 - In api the payload contain $\text{page}=\text{default_page}+3+3+3$
 - Validate UI the page number is $\text{default_page}+3+3+3$
- 7 Click on the first **Page (...)**
- Page ... to be loaded
 - In api the payload contain $\text{page}=\text{default_page}+3+3$
 - Validate UI the page number is $\text{default_page}+3+3$

0016 - Verify Previous and Next button page Functionality

Priority: High

Step	Action	Expected Result
p		

- | | | |
|---|--------------------------------|---|
| 1 | Open application | Page 1 is displayed
In api the payload contain <code>page=1</code> |
| 2 | Previous button to be disabled | Validate previous button is disabled |
| 4 | Click Next button | <ul style="list-style-type: none">• Validate UI the selected page number is <code>default_page_no +1</code> |
| 5 | Go to Last page | <ul style="list-style-type: none">• Wait to page to load |
| 6 | Next button to be disabled | Validate Next button is disabled |
| 7 | Click Previous button | <ul style="list-style-type: none">• Validate UI the selected page number is <code>last_page_no - 1</code> |

0017 - Direct Navigation to All Category

Type: Negative / Known Issue

Test Data

<https://tmdb-discover.surge.sh/popular>

Steps

Step	Action	Expected Result
1	Open the base application	Application loads
2	Open <code>/popular</code> directly in a new tab	Application should ideally resolve the route
3	Open <code>/trend</code> directly in a new tab	Application should ideally resolve the route
4	Open <code>/new</code> directly in a new tab	Application should ideally resolve the route
5	Open <code>/top</code> directly in a new tab	Application should ideally resolve the route
6	Record actual behavior	Any known routing failure should be documented
7	Verify user can recover by opening base URL	Base application should remain accessible

Defect Classification

Type: Routing / Deep-link issue

Severity: Medium

Priority: Medium

depending on product requirements.